---------- Slip 11 ----------

Q.1) Write a C program to get and set the resource limits such as files, memory associated with a Process

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/resource.h>
#include <unistd.h>

void print_limit(const char* name, struct rlimit* limit) {
    printf("%s: soft limit = %ld, hard limit = %ld\n", name, limit->rlim_cur, limit->rlim_max);
}

int main() {
    struct rlimit rl;

    // 1. Get the current limit of maximum number of open files (RLIMIT_NOFILE)
    if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
        perror("getrlimit RLIMIT_NOFILE");
        exit(1);
    }
    print_limit("Open files", &rl);

    // 2. Set a new soft limit for open files
    rl.rlim_cur = 1024; // new soft limit
    // rl.rlim_max can be kept same
    if (setrlimit(RLIMIT_NOFILE, &rl) == -1) {
        perror("setrlimit RLIMIT_NOFILE");
    } else {
        printf("Soft limit for open files updated successfully.\n");
    }

    // Verify new limits
    if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
        perror("getrlimit RLIMIT_NOFILE");
```

```
        exit(1);
    }
    print_limit("Open files", &rl);

    // 3. Get and set memory limit (virtual memory RLIMIT_AS)
    if (getrlimit(RLIMIT_AS, &rl) == -1) {
        perror("getrlimit RLIMIT_AS");
        exit(1);
    }
    print_limit("Virtual memory", &rl);

    // Set new soft limit for memory (example: 256 MB)
    rl.rlim_cur = 256 * 1024 * 1024;
    if (setrlimit(RLIMIT_AS, &rl) == -1) {
        perror("setrlimit RLIMIT_AS");
    } else {
        printf("Soft limit for virtual memory updated successfully.\n");
    }

    // Verify new memory limits
    if (getrlimit(RLIMIT_AS, &rl) == -1) {
        perror("getrlimit RLIMIT_AS");
        exit(1);
    }
    print_limit("Virtual memory", &rl);

    return 0;
}
```

Q.2) Write a C program that redirects standard output to a file output.txt. (use of dup and open system call).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
   int fd;

   // Open (or create) output.txt for writing
   fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
   if (fd < 0) {
      perror("open");
      exit(1);
   }

   // Duplicate fd to standard output (stdout)
   if (dup2(fd, STDOUT_FILENO) < 0) {
      perror("dup2");
      exit(1);
   }

   close(fd); // Close original fd as it's no longer needed

   // Any printf statements now go to output.txt
   printf("This is a message redirected to output.txt\n");
   printf("Hello! Using dup and open to redirect stdout.\n");

   return 0;
}
```